MLP Viva: Additional Questions & Answers — Part 2

More unique questions extracted from all student viva experiences across Level 1, 2 & 3

GENERAL & INTRODUCTION QUESTIONS

Q1. How should you introduce yourself in a viva?

Keep it concise (1–2 min): your name, academic background (degree, year), why you chose MLP, and one sentence on what interested you most in the project. End with readiness to walk through your notebook.

Q2. What would you do if you had more time on the project?

Mention concrete improvements: try more feature engineering (e.g., interaction terms, domain-specific features), experiment with additional models (StackingClassifier, Neural Networks), perform more thorough EDA, use automated HPT tools (Optuna/Bayesian optimisation), perform error analysis on misclassified samples, or tune TF-IDF parameters more rigorously.

Q3. What resources did you use for this project?

Acceptable answers: sklearn documentation, Kaggle notebooks/discussions, YouTube tutorials, MLT/MLF course material, research papers. Avoid saying you just used AI/LLM — mention you used it as a helper to understand concepts, not to copy code blindly.

Q4. What other approaches did you try that did not work?

Be honest. E.g., 'I tried adding polynomial features but it caused overfitting. I tried CountVectorizer before TF-IDF but TF-IDF gave better results because it down-weights common words. I tried StandardScaler on skewed data and it did not help, so I switched to RobustScaler.'

Q5. What is your Kaggle rank/leaderboard score?

Know your exact public and private leaderboard score, your rank among participants, and which version of your notebook achieved the best score. Also know what specific change (model, feature, HPT) gave the biggest improvement.

EDA — DEEPER QUESTIONS

Q1. What is skewness and how does it affect your model?

Skewness measures the asymmetry of a distribution. Right-skewed (positive): long tail to the right, mean > median. Left-skewed (negative): long tail to the left. Skewed distributions can hurt linear models and distance-based models. Solutions: log transform, square-root transform, Box-Cox transform to make it more normal.

Q2. What is a word cloud and where can it be used?

A word cloud is a visual where word size is proportional to its frequency in a text corpus. Used in NLP EDA to quickly identify the most common words in each class or the overall dataset. It helps spot domain-specific terms, stop words that should be removed, and any data quality issues.

Q3. Why use both seaborn and matplotlib? What is the difference?

Matplotlib is the foundational plotting library — low-level, fully customisable, but verbose. Seaborn is built on top of matplotlib — higher-level, produces statistically-oriented plots (heatmaps, violin, pair plots) with less code and better default aesthetics. Use seaborn for quick statistical plots, matplotlib for fine-grained customisation.

Q4. What does .describe() tell you?

df.describe() returns count, mean, std, min, 25th percentile (Q1), 50th percentile (median), 75th percentile (Q3), and max for each numeric column. It gives a quick statistical summary to spot outliers (compare min/max to Q1/Q3), check for missing values (count < total rows), and understand the scale of each feature.

Q5. What is the IQR method for outlier detection?

IQR = Q3 – Q1. Lower bound = Q1 – 1.5 × IQR. Upper bound = Q3 + 1.5 × IQR. Values outside these bounds are considered outliers. The 1.5 factor is standard (Tukey's rule); using 3.0 detects only extreme outliers.

Q6. What is mean and standard deviation?

Mean = (sum of all values) / n — the average, sensitive to outliers. Standard deviation σ = sqrt(mean of squared deviations from mean) — measures spread. A small σ means data is clustered around the mean; large σ means data is spread out. Used in StandardScaler: z = (x – mean) / σ.

Q7. What is Pearson vs. Spearman correlation?

Pearson measures linear correlation between two continuous variables, assuming normality. Range: [–1, 1]. Spearman measures monotonic (rank-based) correlation — does not assume normality, works for ordinal data and non-linear monotonic relationships. Use Spearman when data is skewed or ordinal.

Q8. How do you find correlation between a continuous and a categorical feature?

Use point-biserial correlation (for binary categorical), ANOVA F-test (for multi-class categorical vs. continuous), or Eta-squared. For two categorical features: Chi-Square test or Cramér's V. These tell you whether the categorical grouping significantly differs in the numeric feature's distribution.

PREPROCESSING — DEEPER QUESTIONS

Q1. What is the difference between fillna() and SimpleImputer?

fillna() is a pandas method for quick filling — it does not fit/transform and does not integrate with sklearn Pipelines. SimpleImputer is a sklearn transformer that fits (learns mean/median/mode from training data) and transforms — it integrates seamlessly with Pipeline and ColumnTransformer, preventing data leakage.

Q2. Why should you avoid manual preprocessing (fillna, manual encoding) in favour of Pipelines?

Manual preprocessing applied to the full dataset before splitting causes data leakage (test data info leaks into training). Pipelines enforce that fitting happens only on training data and transformation is consistently applied to both train and test. They also make the code reproducible, deployable, and easier to tune with GridSearchCV.

Q3. What is handle_unknown='ignore' in OneHotEncoder?

When the encoder encounters a category at inference time that it did not see during training, handle_unknown='ignore' outputs a zero vector for that category (all OHE columns = 0) instead of raising an error. This is essential for production systems where new categories may appear.

Q4. What is sparse_output=False in OneHotEncoder?

By default, OHE returns a sparse matrix (efficient memory storage with mostly zeros). sparse_output=False returns a dense numpy array, which is required by some downstream estimators or when you want to inspect the data as a regular array. In newer sklearn versions this parameter is set_output(transform='pandas').

Q5. What is forward fill (ffill) and backward fill (bfill) in imputation?

Used for ordered/time-series data. Forward fill propagates the last known value forward to fill NaN. Backward fill uses the next known value to fill backward. Appropriate when data has temporal order and nearby values are likely similar. Not suitable for randomly missing tabular data.

Q6. What does class_weight='balanced' do in sklearn models?

It automatically adjusts class weights inversely proportional to class frequencies: weight for class i = n_samples / (n_classes × n_samples_in_class_i). This penalises misclassification of the minority class more heavily, effectively oversampling the minority class in the loss function — useful for imbalanced datasets.

Q7. What does stratify=y do in train_test_split?

It ensures the train and test splits preserve the same class distribution as the original dataset. Without stratify, random splits may produce very different class ratios (especially with small or imbalanced datasets), leading to biased evaluation.

Q8. What is the validation size in train_test_split?

test_size parameter: e.g., test_size=0.2 means 20% goes to test (validation), 80% to training. Common splits: 80/20 or 70/30. A larger test set gives more reliable evaluation but less data for training. With stratify, both splits have the same class proportions.

FEATURE ENGINEERING — DEEPER QUESTIONS

Q1. What are lag features and rolling features in time series?

Lag features: the value of a variable at a previous time step (e.g., sales_lag1 = previous day's sales). They capture autocorrelation (today's value depends on yesterday's). Rolling features: statistics (mean, std, sum) computed over a sliding window (e.g., 7-day rolling average). They smooth noise and capture trends. Both help models detect temporal patterns without needing explicit time-series models.

Q2. What is feature selection vs. feature extraction?

Feature selection chooses a subset of original features (RFE, SelectKBest, mutual information, correlation-based) — features remain interpretable. Feature extraction creates new features from original ones (PCA, LDA, TF-IDF) — original features are transformed into a new space, often losing interpretability.

Q3. What is SelectKBest and how does it work?

SelectKBest selects the top k features based on a scoring function. For classification: chi2 (chi-square, for non-negative features), f_classif (ANOVA F-test), or mutual_info_classif. For regression: f_regression or mutual_info_regression. It ranks features by their statistical relationship with the target and keeps the best k.

Q4. What is mutual information (MI)?

Mutual information measures how much knowing one variable reduces uncertainty about another. MI=0: independent variables. Higher MI: stronger dependency. Unlike correlation, MI captures non-linear relationships. Used in SelectKBest with mutual_info_classif or mutual_info_regression.

Q5. What is TruncatedSVD and how does it differ from PCA?

TruncatedSVD is a linear dimensionality reduction using Singular Value Decomposition, but unlike PCA it does NOT center the data first. This makes it efficient for sparse matrices (like TF-IDF output) where centering would destroy sparsity. PCA = SVD on centered data. TruncatedSVD is also called Latent Semantic Analysis (LSA) in NLP.

HYPERPARAMETER TUNING — DEEPER QUESTIONS

Q1. What does the 'verbose' parameter do in sklearn models/GridSearchCV?

verbose controls the amount of logging output during fitting. verbose=0: silent. verbose=1: minimal (one line per model). verbose=2: one line per fold. Higher values give more detail. Useful for monitoring long HPT runs but should be turned off in production.

Q2. What is the 'scoring' parameter in GridSearchCV? What does scoring='f1_macro' mean?

scoring specifies the metric used to evaluate each hyperparameter combination. 'f1_macro' computes F1 for each class independently and takes the unweighted mean — treats all classes equally regardless of support. Other options: 'accuracy', 'roc_auc', 'f1_weighted', 'neg_mean_squared_error' (for regression, negative because sklearn maximises). Choose scoring based on your task's priority.

Q3. What happens if you increase/decrease n_estimators when model is overfitting?

If overfitting: decrease n_estimators (fewer trees = simpler model). Also decrease max_depth, increase min_samples_leaf, increase regularisation (lambda/alpha in XGBoost). If underfitting: increase n_estimators, increase max_depth, decrease min_samples_leaf.

Q4. What happens if learning rate is too high or too low?

Too high: gradient descent overshoots the minimum — loss oscillates or diverges, model may not converge. Too low: convergence is very slow, may get stuck in local minima or require thousands of iterations. Optimal: start with 0.01–0.1 and tune. In boosting, a lower learning rate with more estimators often gives better results.

Q5. What are colsample_bytree and subsample in XGBoost?

subsample: fraction of training samples randomly selected for each tree (e.g., 0.8 = 80%). Prevents overfitting by adding randomness (similar to bagging). colsample_bytree: fraction of features randomly selected for each tree. Both add stochasticity — similar to Random Forest's randomness but within a boosting framework.

Q6. What is num_leaves in LightGBM and how does it differ from max_depth?

LightGBM grows trees leaf-wise (best-first), not level-wise. num_leaves controls the maximum number of leaves in one tree — directly controls model complexity. A higher num_leaves = more complex model. max_depth can constrain depth too, but num_leaves is the primary LightGBM complexity control. Typical values: 20–150.

MACHINE LEARNING MODELS — DEEPER QUESTIONS

Q1. What is a weak learner?

A weak learner is a model that performs only slightly better than random guessing (accuracy slightly above 50% for binary classification). Examples: shallow decision tree (depth=1, called a 'decision stump'), linear model on complex data. Boosting combines many weak learners to form a strong learner.

Q2. What is a baseline model?

A baseline model is the simplest possible model used as a reference point. For classification: predict the majority class always (DummyClassifier). For regression: predict the mean always (DummyRegressor). All subsequent models must outperform the baseline to justify their complexity.

Q3. When is ensemble learning NOT recommended?

1) When you need high interpretability (single decision tree or logistic regression is better). 2) When training data is very small (ensembles may overfit). 3) When inference speed is critical (single lightweight model preferred). 4) When the problem is simple and a single model performs well enough. 5) When compute resources are very limited.

Q4. What is ExtraTreesClassifier (Extremely Randomised Trees)?

Like Random Forest but with additional randomness: both the feature AND the split threshold are chosen randomly (not the best threshold). This makes it faster to train and often reduces variance further, though it may increase bias slightly. It is also a bagging ensemble.

Q5. What is AdaBoost?

Adaptive Boosting trains a sequence of weak learners, giving higher weights to misclassified samples so subsequent learners focus on them. Final prediction is a weighted vote of all learners. It was one of the first successful boosting algorithms. Sensitive to noisy data and outliers (unlike Gradient Boosting which is more robust).

Q6. What is Stochastic Gradient Descent (SGD) vs. Batch Gradient Descent?

Batch GD: uses the entire training set to compute gradients — slow per iteration, guaranteed convergence but expensive. SGD: uses one sample at a time — fast, noisy updates, can escape local minima but may not converge to exact minimum. Mini-batch GD: compromise (small random batches) — fast and relatively stable, used in deep learning. sklearn's SGDClassifier uses SGD.

Q7. What is min_samples_split vs. min_samples_leaf?

min_samples_split: minimum number of samples required to split an internal node. If a node has fewer samples, it becomes a leaf. min_samples_leaf: minimum number of samples required in a leaf node after splitting. Both prevent overfitting. E.g., min_samples_split=8 means a node with 7 samples will NOT split (becomes a leaf).

Q8. What is the 'degree' parameter in SVC (Polynomial kernel)?

For the polynomial kernel K(x,y) = (γ·x·y + r)^d, degree d controls the complexity of the decision boundary. degree=2: quadratic boundary. degree=3: cubic. Higher degree = more complex = more risk of overfitting. Used only when kernel='poly'.

Q9. What is Naive Bayes and why is it 'Naive'?

Naive Bayes applies Bayes' theorem (P(class|features) ∝ P(features|class)·P(class)) with the 'naive' assumption that all features are conditionally independent given the class. This is rarely true in practice but simplifies computation dramatically. Despite the unrealistic assumption, it works well for text classification (spam detection, sentiment analysis).

Q10. What is the difference between likelihood and probability?

Probability: given fixed parameters, what is the chance of observing this data? P(data|parameters). Likelihood: given observed data, how likely are different parameter values? L(parameters|data). Likelihood is used in Maximum Likelihood Estimation (MLE) to fit model parameters.

Q11. What is the difference between KMeans (clustering) and KNN (classification/regression)?

KMeans is unsupervised — groups unlabelled data into k clusters based on feature similarity (minimises inertia). KNN is supervised — predicts labels for new points based on the k nearest labelled training points. KMeans learns cluster centroids; KNN stores all training data (lazy learner). Different k: KMeans k = number of clusters; KNN k = number of neighbours.

Q12. What is Ridge (L2) and Lasso (L1) Regression?

Ridge Regression adds L2 penalty (λΣw■²) to linear regression's loss function. Shrinks all coefficients towards zero but rarely eliminates them — good for multicollinearity. Lasso Regression adds L1 penalty (λΣ|w■|). Can shrink coefficients exactly to zero (sparse model), acting as automatic feature selection. Use Lasso when you believe many features are irrelevant.

Q13. Can Logistic Regression be used for multi-class problems? How?

Yes. Two strategies: One-vs-Rest (OvR/OvA): train one binary classifier per class (k classifiers for k classes). One-vs-One (OvO): train a classifier for each pair of classes. Multinomial (softmax): extends logistic regression directly to k classes using the softmax function. In sklearn, multi_class='multinomial' or 'ovr'.

Q14. Which model is better for outliers — Logistic Regression or SVM?

SVM (with RBF kernel) is generally more robust to outliers because the decision boundary is determined only by support vectors (points near the margin). Logistic Regression is sensitive to outliers as they can heavily influence the decision boundary. However, with a small C (strong regularisation), SVM can also be affected.

Q15. What are tree-based algorithms? Name them.

Decision Tree, Random Forest (bagging of DTs), Extra Trees, AdaBoost (boosting with DT stumps), Gradient Boosting (sklearn), XGBoost, LightGBM, CatBoost. All are non-parametric, handle non-linearity naturally, and are relatively robust to outliers and feature scaling (no scaling needed).

Q16. What is a parametric vs. non-parametric model?

Parametric: assumes a fixed functional form and learns a fixed number of parameters regardless of data size. Examples: Logistic Regression, Linear Regression, Naive Bayes, Neural Networks (fixed architecture). Non-parametric: makes fewer assumptions; model complexity can grow with data size. Examples: KNN, Decision Trees, Random Forest, SVM (with kernel).

Q17. What is the difference between Linear Regression and Logistic Regression?

Linear Regression: predicts continuous outputs; uses MSE loss; output is unbounded (–∞ to +∞). Logistic Regression: predicts class probabilities; uses log-loss (cross-entropy); applies sigmoid to bound output to [0,1]; used for classification. Both use a linear combination of features, but logistic regression applies a non-linear sigmoid transformation.

Q18. What is the OOB (Out-of-Bag) score in Random Forest?

Since each tree in a Random Forest is trained on a bootstrap sample (~63% of data), the remaining ~37% (out-of-bag samples) can be used as a validation set for that tree without a separate hold-out. The OOB score aggregates predictions across all trees for each sample's out-of-bag instances, giving a free estimate of generalisation performance.

EVALUATION METRICS — DEEPER QUESTIONS

Q1. What is Log Loss and why use it instead of MSE for classification?

Log Loss (Binary Cross-Entropy) = –(1/n)·Σ[y■·log(p■) + (1–y■)·log(1–p■)]. It penalises confident wrong predictions much more heavily than less confident ones (logarithmic penalty). MSE does not correctly penalise probabilistic outputs for classification — it creates a non-convex loss surface making optimisation harder and does not account for the probabilistic nature of class outputs.

Q2. What is hinge loss (used in SVM)?

Hinge loss = max(0, 1 – y·f(x)) where y ∈ {–1, +1} and f(x) is the raw model score. If the correct class is predicted with confidence ≥ 1, loss = 0. Otherwise, loss grows linearly. It is used in SVM with LinearSVC or SGDClassifier(loss='hinge'). Unlike log loss, it does not output probabilities.

Q3. What is F2-score?

$F_\beta = (1 + \beta^2) \times (\text{Precision} \times \text{Recall}) / (\beta^2 \cdot \text{Precision} + \text{Recall})$. F2 (β=2) weights Recall twice as much as Precision. Use when false negatives are twice as costly as false positives — e.g., in medical diagnosis where missing a disease (FN) is worse than a false alarm (FP).

Q4. What is SMAPE?

Symmetric Mean Absolute Percentage Error = (100/n)·Σ |actual–predicted| / ((|actual| + |predicted|)/2). It is a scale-independent percentage error metric symmetric between over- and under-estimation. Used in some forecasting competitions. Range: 0–200%.

Q5. Precision vs. Recall: which one to use for poisonous apple detection?

Use Recall (Sensitivity). In poisonous apple detection, a False Negative (predicting a poisonous apple as safe) could kill someone — this is far worse than a False Positive (flagging a safe apple as poisonous). Recall = TP/(TP+FN) directly measures the proportion of actual positives caught. Maximise Recall to minimise dangerous misses.

Q6. What is Precision vs. Recall for fraud detection?

In fraud detection: Recall matters most — missing a fraudulent transaction (FN) is costly. But very low Precision means too many false alarms (FP), annoying customers. F1 or Fβ (β>1) balances both. AUC-ROC or PR-AUC is also commonly used since fraud data is heavily imbalanced.

Q7. What are the formulas for Precision, Recall, Accuracy, and F1?

Accuracy = (TP + TN) / (TP + TN + FP + FN) Precision = TP / (TP + FP) Recall = TP / (TP + FN) F1 = 2 × Precision × Recall / (Precision + Recall) = 2TP / (2TP + FP + FN)

NEURAL NETWORKS

Q1. What is a neural network and how does it work?

A neural network is a series of layers: input layer (receives features), hidden layers (learn intermediate representations via weighted sums + activation functions), output layer (produces final prediction). Training uses forward propagation (compute predictions) then backpropagation (compute gradients via chain rule) and gradient descent to update weights. Each neuron computes: output = activation(Σ w■x■ + b).

Q2. How many hidden layers can a neural network have?

Theoretically unlimited. A single hidden layer can approximate any continuous function (Universal Approximation Theorem). In practice: 1–2 hidden layers work for most tabular data tasks. Deep networks (many layers) are used for image, text, and audio. More layers = more representational power but harder to train (vanishing gradients, longer training).

Q3. What happens if you increase the number of hidden layers?

The model becomes more complex and can learn more abstract features. Benefits: better performance on complex tasks (images, NLP). Risks: vanishing/exploding gradients, slower training, higher risk of overfitting, requires more data and regularisation (dropout, batch normalisation).

Q4. What is gradient vanishing?

During backpropagation, gradients are multiplied through each layer via the chain rule. With many layers and sigmoid/tanh activations (which saturate near 0 or 1), gradients become exponentially small as they propagate back — early layers learn very slowly. Solutions: ReLU activation (does not saturate for positive values), batch normalisation, residual connections (skip connections), gradient clipping.

Q5. What are optimisation functions (optimisers) in neural networks?

SGD (Stochastic Gradient Descent): basic gradient update. Momentum: accumulates past gradients to accelerate. RMSprop: adapts learning rate per parameter using running average of squared gradients. Adam (Adaptive Moment Estimation): combines Momentum + RMSprop — most popular default. AdaGrad: accumulates all past squared gradients (learning rate decreases over time).

Q6. What is a generative vs. discriminative model?

Discriminative model: directly models the boundary between classes P(y|X). Examples: Logistic Regression, SVM, Neural Networks, Decision Trees. Generative model: models the joint distribution P(X,y) or P(X|y) and P(y), then uses Bayes' theorem. Examples: Naive Bayes, Gaussian Mixture Models, GANs. Logistic Regression is discriminative despite being named 'regression.'

NLP — DEEPER QUESTIONS

Q1. What is the difference between CountVectorizer and TF-IDF?

CountVectorizer simply counts how many times each word appears in a document (raw term frequency). TF-IDF = TF × IDF — it down-weights common words (high document frequency) and up-weights rare, informative words. TF-IDF generally gives better results for text classification because common words like 'the' or 'is' get penalised and do not dominate.

Q2. What is BERT and when would you use it?

BERT (Bidirectional Encoder Representations from Transformers) is a pre-trained transformer-based language model. It understands context bidirectionally (unlike TF-IDF which is bag-of-words). Use BERT for complex NLP tasks (sentiment analysis, NER, Q&A;) when you have sufficient compute and data. For simple classification with limited compute, TF-IDF + traditional ML is often preferred.

Q3. What is an ARIMA model and what are its parameters?

ARIMA (AutoRegressive Integrated Moving Average) is a time series model. Parameters: p (AutoRegressive order — number of lag observations), d (differencing order — number of times series is differenced to make it stationary), q (Moving Average order — number of lagged forecast errors). SARIMA extends ARIMA with seasonal components (P, D, Q, s).

Q4. What is time series cross-validation (TSCV)?

Standard k-fold CV randomly shuffles data — invalid for time series (future data leaks into training). Time Series CV (walk-forward validation) preserves temporal order: train on past, validate on future. Each fold expands the training set forward in time. sklearn: TimeSeriesSplit.

STATISTICS FOR ML

Q1. What is a t-test?

A t-test determines whether the means of two groups are statistically significantly different. One-sample t-test: compare sample mean to a known value. Independent samples t-test: compare means of two independent groups. Paired t-test: compare means of the same group at two time points. The test statistic follows a t-distribution under the null hypothesis.

Q2. What is a Chi-Square test and when is it used in ML?

Chi-Square (χ^2) test of independence checks whether two categorical variables are statistically independent. In ML: used in SelectKBest(chi2) to select features with the strongest chi-square statistic with the target. Also used in EDA to confirm that categorical features are relevant to the target. Requires non-negative feature values.

Q3. What is p-value and alpha level?

p-value: probability of observing results at least as extreme as the data, assuming the null hypothesis is true. If $p < \alpha$ (significance level, commonly 0.05), we reject the null hypothesis. $\alpha = 0.05$ means we accept a 5% chance of false rejection (Type I error). In feature selection, low p-value → feature is significantly associated with target → keep it.

Q4. What is Pearson correlation formula?

r = Σ[(x■ – x■)(y■ – ■)] / [sqrt(Σ(x■–x■)²) × sqrt(Σ(y■–■)²)]. Range: [–1, +1]. r = +1: perfect positive linear relationship. r = –1: perfect negative linear relationship. r = 0: no linear relationship (may still have non-linear relationship).

Q5. What is normal distribution (Gaussian distribution)?

A symmetric bell-shaped distribution defined by mean μ and standard deviation σ. 68% of data falls within μ±σ, 95% within μ±2σ, 99.7% within μ±3σ (empirical rule). Many natural phenomena follow this distribution. Assumed by linear/logistic regression, StandardScaler works best on normally distributed features.

SCENARIO & CASE STUDY QUESTIONS

Q1. House price prediction: which model would you choose and explain its steps?

This is a regression problem (predicting continuous price). Best choices: Gradient Boosting (XGBoost/LightGBM) for accuracy, or Ridge/Lasso if interpretability is needed. Steps for linear regression: 1) Collect features (area, location, age, bedrooms). 2) Handle missing values. 3) Encode categoricals. 4) Scale numerics. 5) Train on historical data. 6) Evaluate with RMSE/R2. 7) Predict future price. For time-based prediction, also consider ARIMA or adding time-based features.

Q2. Fraud transaction detection: how would you approach it?

1) Frame as binary classification (fraud/not-fraud). 2) Handle severe class imbalance (fraud is rare) — use SMOTE, class_weight='balanced', or AUC-PR as metric. 3) Feature engineer: transaction amount, time of day, frequency, location deviation. 4) Use tree-based models (XGBoost, Random Forest) or anomaly detection. 5) Evaluate with Recall (minimise missed fraud) and Precision (minimise false alarms). 6) Consider threshold tuning for business requirements.

Q3. Social media company: what recommendations would you give to the owner based on your project?

Based on your toxicity/comment classification model: 1) Flag high-probability toxic comments for human review before publishing. 2) Prioritise moderation resources on comment threads with highest toxic density. 3) Monitor trends by hour/day (temporal analysis). 4) Target specific toxicity types (hate speech, threats) for different escalation workflows. 5) Use model confidence scores to distinguish borderline vs. clear violations.

Q4. If a dataset has 200 features, what would be your approach?

1) Start with EDA — correlation heatmap, variance threshold (remove near-zero variance features). 2) Use feature importance from a quick Random Forest to shortlist. 3) Apply RFE or SelectKBest to systematically rank features. 4) PCA/TruncatedSVD for dimensionality reduction if features are correlated. 5) L1 regularisation (Lasso) in linear models for automatic feature elimination. 6) Cross-validate performance as you reduce features to find the optimal number.

Q5. If training accuracy is very low and test accuracy is also low, what is the problem?

This is underfitting (high bias). The model is too simple to capture the patterns in the data. Solutions: use a more complex model, add more relevant features, do more feature engineering, reduce regularisation, train for more iterations.

Q6. If training accuracy is very high but test accuracy is low, what is the problem?

This is overfitting (high variance). The model memorises the training data but fails to generalise. Solutions: get more training data, reduce model complexity, add regularisation (L1/L2/dropout), use cross-validation, feature selection, early stopping, or bagging.

Q7. What is the case of low training accuracy and high test accuracy?

This is unusual and often indicates data leakage — test data information has leaked into training, making test performance artificially inflated. Or it may indicate the training set is harder/noisier than the test set. Investigate if preprocessing was fit on the full dataset or if there is a train/test distribution mismatch.

Q8. If you are in a company and this is your project, what 4–5 insights or recommendations would you give?

Think like a business analyst: 1) Which features are the strongest predictors of your target? 2) What patterns did EDA reveal (seasonality, demographic trends, outlier groups)? 3) What is the confidence of the model's predictions (use probabilities, not just hard labels)? 4) Where does the model fail (which class/segment has lowest recall)? 5) What business action can be taken for each prediction bucket (high-risk vs. low-risk)?

ADDITIONAL CODING QUESTIONS

Q1. Code to replace '?' values with NaN and handle them

Replace common missing value placeholders and then impute.

```
import pandas as pd
import numpy as np

df = pd.read_csv('data.csv')
df.replace('?', np.nan, inplace=True)

# Check missing counts
print(df.isnull().sum())

# Impute numeric with median
for col in df.select_dtypes(include='number').columns:
    df[col].fillna(df[col].median(), inplace=True)

# Impute categorical with mode
```

```
for col in df.select_dtypes(include='object').columns:
    df[col].fillna(df[col].mode()[0], inplace=True)
```

Q2. Code to build a confusion matrix manually (numpy) and compute precision/recall

Build and interpret confusion matrix without sklearn.

```
import numpy as np

# Example: 4-class confusion matrix
cm = np.array([
    [678, 78, 82, 95],
    [ 75,600, 85, 93],
    [ 45, 89,569, 45],
    [ 13, 78, 65,700]
])

def calculate_precision(class_idx, cm):
    tp = cm[class_idx, class_idx]
    fp = np.sum(cm[:, class_idx]) - tp
    return tp / (tp + fp) if (tp + fp) > 0 else 0

def calculate_recall(class_idx, cm):
    tp = cm[class_idx, class_idx]
    fn = np.sum(cm[class_idx, :]) - tp
    return tp / (tp + fn) if (tp + fn) > 0 else 0

for i in range(cm.shape[0]):
    print(f'Class {i}: Precision={calculate_precision(i,cm):.3f}, Recall={calculate_recal
l(i,cm):.3f}')
```

Q3. Code to load iris dataset, apply PCA and show variance explained

Demonstrate PCA with variance explained before and after.

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import numpy as np

iris = load_iris()
X = iris.data  # 4 features

# Standardize first
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply PCA
pca = PCA()
pca.fit(X_scaled)

print('Explained variance ratio:', pca.explained_variance_ratio_)
print('Cumulative:', np.cumsum(pca.explained_variance_ratio_))

# Keep 2 components
pca2 = PCA(n_components=2)
X_pca = pca2.fit_transform(X_scaled)
```

```
print('Original shape:', X_scaled.shape)
print('After PCA shape:', X_pca.shape)
```

Q4. Code to apply KMeans and find optimal k using elbow method

Find the best k for KMeans clustering.

```
from sklearn.datasets import load_digits
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

digits = load_digits()
X = digits.data

inertias = []
k_range = range(2, 15)
for k in k_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X)
    inertias.append(km.inertia_)

plt.plot(k_range, inertias, 'bo-')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Inertia')
plt.title('Elbow Method')
plt.show()
# Choose k at the 'elbow' — where inertia reduction slows
```

Q5. Code to apply RFE on a dataset and find top 2 features

Full pipeline: load data, clean, apply RFE.

```
import pandas as pd
import numpy as np
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

# Load data (example with '?' missing values)
df = pd.read_csv('data.csv')
df.replace('?', np.nan, inplace=True)
df.dropna(inplace=True)

# Separate features and target
X = df.iloc[:, :-1]
y = df.iloc[:, -1]

# Encode if needed
X = pd.get_dummies(X)

# Scale
X_scaled = StandardScaler().fit_transform(X)

# RFE
model = LogisticRegression(max_iter=1000)
rfe = RFE(estimator=model, n_features_to_select=2)
rfe.fit(X_scaled, y)
```

```
selected = X.columns[rfe.support_]
print('Top 2 features:', selected.tolist())
```

Q6. Code to apply MinMaxScaler

Scale features to [0, 1] range.

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler(feature_range=(0, 1))
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# Note: Only fit on training data, transform both
```

Q7. Code to train ElasticNet and measure performance

ElasticNet combines L1 and L2 regularisation.

```
from sklearn.linear_model import ElasticNet
from sklearn.metrics import r2_score, mean_squared_error

model = ElasticNet(alpha=0.1, l1_ratio=0.5, random_state=42)
# l1_ratio=0: pure Ridge, l1_ratio=1: pure Lasso
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print('R2:', r2_score(y_test, y_pred))
print('RMSE:', mean_squared_error(y_test, y_pred, squared=False))
```

Q8. Code to print columns of a dataset and display unique values/frequencies

Common pandas operations tested in coding rounds.

```
import pandas as pd

df = pd.read_csv('train.csv')

# All column names
print(df.columns.tolist())

# Unique values in a column
print(df['religion'].unique())

# Value counts (frequency)
print(df['label'].value_counts())

# Null values per column
print(df.isnull().sum())

# Replace None and NaN with 'other'
df['religion'] = df['religion'].replace({None: 'other', 'none': 'other'})
df['religion'].fillna('other', inplace=True)
```

Q9. Code a simple logistic regression from scratch using NumPy

Implements forward pass and gradient update.

```
import numpy as np

class LogisticRegressionScratch:
```

```
    def __init__(self, lr=0.01, epochs=1000):
        self.lr = lr
        self.epochs = epochs

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def fit(self, X, y):
        self.w = np.zeros(X.shape[1])
        self.b = 0
        for _ in range(self.epochs):
            z = X @ self.w + self.b
            y_hat = self.sigmoid(z)
            # Gradients
            dw = (1/len(y)) * X.T @ (y_hat - y)
            db = (1/len(y)) * np.sum(y_hat - y)
            self.w -= self.lr * dw
            self.b -= self.lr * db

    def predict(self, X):
        return (self.sigmoid(X @ self.w + self.b) >= 0.5).astype(int)
```

Q10. Code to create a heatmap between two encoded categorical columns

Encode categoricals, then plot correlation heatmap.

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
import seaborn as sns
import matplotlib.pyplot as plt

# Encode V1, V2
le = LabelEncoder()
df['V1_enc'] = le.fit_transform(df['V1'].astype(str))
df['V2_enc'] = le.fit_transform(df['V2'].astype(str))

# Correlation heatmap
corr = df[['V1_enc', 'V2_enc']].corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm')
plt.show()
```

MISCELLANEOUS & TRICKY QUESTIONS

Q1. What is the difference between Pipeline and ColumnTransformer?

Pipeline: chains steps sequentially — each step's output becomes the next step's input. Used for: imputer → scaler → model (linear sequence). ColumnTransformer: applies different transformers to different columns IN PARALLEL and concatenates the results. Used for: apply num_pipeline to numeric cols and cat_pipeline to categorical cols simultaneously. In practice: ColumnTransformer is nested inside a Pipeline as the preprocessing step.

Q2. What does axis=0 vs. axis=1 mean in pandas?

axis=0 operates along rows (down the columns). E.g., df.drop('row_label', axis=0) drops a row; df.mean(axis=0) computes column-wise mean. axis=1 operates along columns (across rows). E.g., df.drop('col_name', axis=1) drops a column; df.sum(axis=1) sums across columns for each row.

Q3. How do you convert a sparse matrix (sklearn output) to a pandas DataFrame?

With newer sklearn (>=1.2): set set_output(transform='pandas') on the transformer. Or: pd.DataFrame.sparse.from_spmatrix(sparse_matrix, columns=feature_names). Or simply: pd.DataFrame(sparse_matrix.toarray(), columns=feature_names). Note: toarray() may be memory-intensive for very large matrices.

Q4. Did PCA improve your model score? Why or why not?

PCA may not improve score when: the dataset is small (PCA reduces information), the features are already low-dimensional, or tree-based models are used (they are not affected by scale/correlation). PCA helps when: there are many correlated features, the model benefits from reduced dimensionality (linear models, KNN, SVM), or training speed is a concern. Always compare performance with and without PCA using cross-validation.

Q5. What is the difference between load_* and fetch_* functions in sklearn?

load_* (e.g., load_iris): loads a small dataset bundled with sklearn from local memory — fast, no internet needed. fetch_* (e.g., fetch_20newsgroups, fetch_openml): downloads larger datasets from the internet on first call, then caches locally. If you use load_ when fetch_ is needed, you will get an error.

Q6. What is the StackingClassifier?

Stacking (stacked generalisation) trains multiple base models (level-0 learners) and feeds their predictions as features to a meta-learner (level-1 learner). Unlike bagging (parallel, same algorithm) or boosting (sequential), stacking combines diverse algorithms. The meta-learner learns how to best combine base model predictions. Can be powerful but risks overfitting if not done with cross-validated predictions.

Q7. What is SelectKBest vs. RFE?

SelectKBest: scores each feature independently using a statistical test (chi2, f_classif, mutual information) — fast, ignores feature interactions. RFE: iteratively trains a model and removes the least important feature — captures feature interactions via model importance but is slower. Use SelectKBest for quick filtering; RFE for model-aware selection.

Q8. What is the 'support' vector in SVM and why does it matter?

Support vectors are the training data points closest to the decision boundary (within or on the margin). They are the only points that define the hyperplane — all other points do not matter. This makes SVM memory-efficient and robust: adding points far from the boundary does not change the model. However, moving support vectors drastically changes the decision boundary.

Q9. What is collinearity and how do you handle it?

Collinearity (multicollinearity) occurs when two or more features are highly correlated with each other (not necessarily with the target). It inflates variance of coefficient estimates in linear models, making them unstable and hard to interpret. Detection: correlation matrix, Variance Inflation Factor (VIF). Solutions: drop one of the correlated features, use PCA, or use Ridge Regression (L2 regularisation).

Q10. What is the real name of a box plot?

Box-and-whisker plot (coined by statistician John Tukey). The 'box' shows the IQR (Q1 to Q3), the line inside is the median, and the 'whiskers' extend to 1.5×IQR. Points beyond whiskers are plotted individually as outliers.